

Assignment 1: Mini Project – Console Record-Management Application Course: MCA
Semester I – Python Programming & Relational Database
Submitted To: Prof. Rohan Gupta
Institution: ITM Skills University, School of FutureTech
Submitted By: Shubham Shah
GitHub Link: https://github.com/shubham112-bip/Student_Management_System

1. Introduction

The Student Management System is a menu-driven Python application designed to handle student data entry, storage, and retrieval efficiently. The objective of this mini-project is to demonstrate a practical understanding of core Python programming concepts, transitioning from theoretical knowledge to building a persistent, error-free CRUD (Create, Read, Update, Delete) application.

2. Objectives

- To develop a modular, function-based Python application for managing student records.
- To provide an interactive Menu-Driven console interface for users.
- To implement File I/O operations ensuring that student data persists across different application sessions.
- To safeguard application execution against runtime and logical errors using Exception Handling.

3. Core Python Concepts Implemented

- **Data Types and Structures:** The application processes multiple data types (strings for student names/courses, integers for Roll Numbers/IDs). It utilizes Python structures like lists and dictionaries to temporarily hold student records in memory before writing them to the storage file.
- **Conditionals and Loops:** A `while` loop maintains the main menu's active state until the user explicitly chooses to exit. `if/elif/else` conditional blocks route the user's menu choice to the corresponding CRUD function.
- **Modular Functions:** Instead of monolithic code, operations are broken down into descriptive functions (e.g., `add_student()`, `view_students()`, `search_student()`, `update_student()`, `delete_student()`). This enhances code readability and reusability.
- **Exception Handling:** `try-except` blocks are heavily utilized in user input sections to prevent crashes. For example, it catches `ValueError` if a user types a letter when an integer Roll Number is expected.
- **File I/O:** The built-in `open()` function is utilized with various file modes (`'r'`, `'w'`, `'a'`). The application reads the file on startup to load existing data and writes to the file upon adding, updating, or deleting student records so no data is lost.

4. System Architecture & Execution Flow

1. **Initialization:** The program attempts to open the text/CSV file containing the student database. If it does not exist, exception handling creates a new, empty file automatically.
2. **Main Menu Display:** The user is presented with a list of options (Add Student, View Students, Search Student, Update Student, Delete Student, Exit).
3. **User Action:** The system prompts the user to input a choice.
4. **Execution:** Based on the choice, the specific function is invoked to handle the logic.
5. **Data Committal:** Any changes made by the user are immediately reflected and saved in the local file.
6. **Return:** The flow returns to the Main Menu until the user selects 'Exit'.

5. File Management & Error Handling Mechanism

Data loss is prevented by writing data permanently to the local storage file. To ensure a seamless user experience, the following error handlers are built-in:

- **FileNotFoundError:** Handled gracefully by creating a fresh database file on the very first run.
- **ValueError:** Caught during data entry to prompt the user to re-enter valid data types without crashing the console.
- **Index & Logic Errors:** Handled safely when a user searches for, updates, or attempts to delete a student record that does not exist in the database.

6. Conclusion

This project successfully integrates fundamental Python operations into a single, functional utility. It demonstrates the ability to manage application state, handle user inputs securely, and structure code modularly in a way that meets standard software development practices.

7. Output Screenshots

(Please insert your screenshots below this line before submitting)

1. Screenshot 1: Application Main Menu

```
===== Student Management System =====
1. Add Student Record
2. View All Student Records
3. Search Student Record
4. Update Student Record
5. Delete Student Record
6. Exit
=====
Enter your choice (1-6):
```

Caption

2. Screenshot 2: Adding a new Student Record successfully

```
Enter your choice (1-6): 1

--- Add Student Record ---
Enter Roll Number: 106
Enter Name: Tanya Singh
Enter Age: 21
Enter Email: tanya.singh@example.com
Enter Phone: 9811223344

Student record added successfully! (Roll No: 106)
```

Caption

3. Screenshot 3: Viewing all Student Records

```
choice (1-6): 2

Student Records ---
-----
1 | Name: Shubham Shah      | Age: 22 | Email: shubhamshah20@gmail.com | Phone: 7208410291
2 | Name: Priya Sharma     | Age: 21 | Email: priya.sharma@example.com | Phone: 9820123456
3 | Name: Aarav Patel      | Age: 22 | Email: aarav.patel@example.com  | Phone: 9876543210
4 | Name: Ananya Verma     | Age: 20 | Email: ananya.verma@example.com  | Phone: 9123456780
5 | Name: Rohan Kulkarni   | Age: 23 | Email: rohan.k@example.com       | Phone: 9988776655
-----
Prompts: 5
```

Caption

4. Screenshot 4: Updating or Deleting a Student Record

```
Enter your choice (1-6): 5

--- Delete Student Record ---
Enter Roll Number of student to delete: 105

Target Record: Roll No: 105 | Name: Rohan Kulkarni | Age: 23 | Email: rohan.k@example.com
Are you sure you want to delete this record? (y/n): y

Student record deleted successfully!
```

Caption

```
Enter your choice (1-6): 4

--- Update Student Record ---
Enter Roll Number of student to update: 105

Current Details: Roll No: 105   | Name: Rohan Kulkarni   | Age: 23   | Email: rohan.k@example.co
Press Enter to keep current value.

Name [Rohan Kulkarni]:
Age [23]: 24
Email [rohan.k@example.com]:
Phone [9988776655]:

Student record updated successfully!
```

Caption

5. Screenshot 5: Error handling (e.g., entering invalid data type for a Roll Number)

```
Enter your choice (1-6): 1

--- Add Student Record ---
Enter Roll Number: abc
Error: Invalid input! Roll Number must be an integer.
```

Caption